

Clase 2: Programas, variables y tipos de datos

Recordatorio de conceptos fundamentales

Antes de comenzar a construir programas, es importante recordar algunos conceptos vistos en la clase anterior.

Un **programa** es un conjunto organizado de instrucciones que una computadora puede ejecutar para realizar una tarea determinada. Estas instrucciones suelen escribirse utilizando un lenguaje de programación.

Estas instrucciones siguen una **lógica previamente definida**, conocida como **algoritmo**.

Un **algoritmo** es una secuencia finita y ordenada de pasos que permiten resolver un problema o realizar una tarea.

En este apunte retomaremos estos conceptos fundamentales y profundizaremos en su aplicación práctica. Además, comenzaremos a ver cómo **diseñar y representar algoritmos** utilizando **diagramas de flujo**, una herramienta que permite visualizar de manera clara la secuencia de instrucciones que componen un programa.

Representación de algoritmos mediante diagramas de flujo

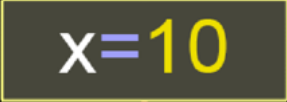
Antes de escribir un programa en un lenguaje de programación, muchas veces resulta útil **representar el algoritmo de forma gráfica**.

Una de las formas más comunes de hacerlo es mediante **diagramas de flujo**.

Un **diagrama de flujo** es una representación gráfica de un algoritmo. Su objetivo es mostrar de forma visual la secuencia de instrucciones que componen un proceso o programa.

Mediante el uso de **símbolos estandarizados y flechas**, los diagramas de flujo permiten representar las diferentes acciones que se realizan dentro de un algoritmo y el orden en que estas se ejecutan. De esta manera, es posible visualizar con mayor claridad cómo fluye la información a lo largo del proceso y cómo se relacionan las distintas operaciones entre sí.

Los elementos básicos de un diagrama de flujo son los siguientes:

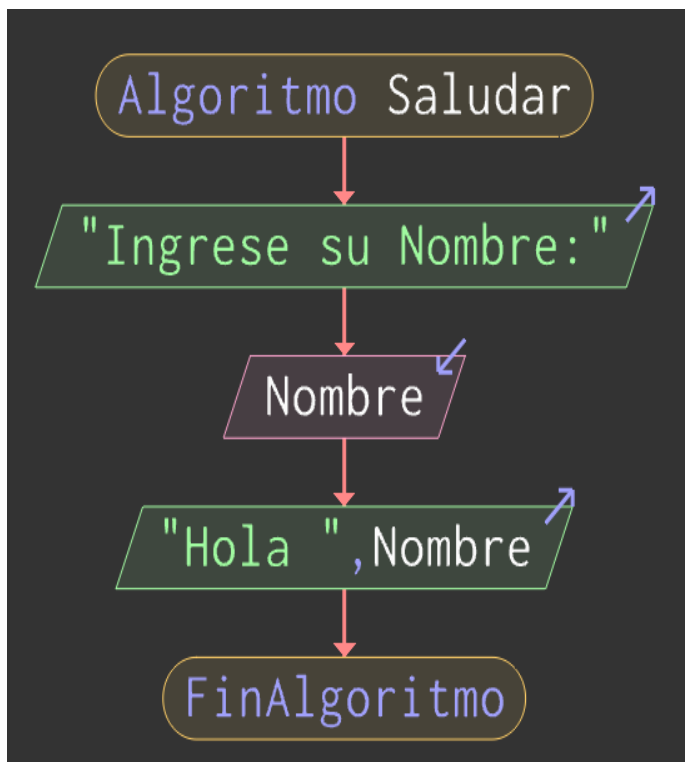
Simbolo	Nombre	Funcion
	Inicio/Fin	Indica el inicio o fin de un algoritmo.
	Linea de flujo	Muestra el orden de operación del proceso. Una línea que viene de un símbolo y apunta a otro.
	Proceso	Representa un conjunto de operaciones que cambian el valor, la forma o la ubicación de los datos.
	Entrada/Salida	Indica el proceso de entrada y salida de datos, como entrada de datos o visualización de resultados.

Los diagramas de flujo permiten visualizar con claridad **el orden de ejecución de las instrucciones**, lo cual facilita la comprensión y el diseño de algoritmos.

Utilizando estos elementos es posible construir algoritmos que permitan resolver distintos tipos de problemas mediante un programa.

Ejemplo: Elaborar un programa que nos pida nuestro nombre y nos salude.

La secuencia de ejecución es la siguiente:



1) El algoritmo comienza con el símbolo de **inicio**.

2) Se **muestra** en pantalla un mensaje solicitando al usuario que ingrese su nombre.

3) El usuario **ingresa** su nombre mediante el teclado, y este valor se almacena en una **variable**.

4) El programa muestra en pantalla un mensaje de saludo que incluye el nombre ingresado por el usuario.

5) Finalmente, el algoritmo llega al símbolo de **fin**, indicando la finalización del programa.

Este ejemplo permite observar cómo un algoritmo organiza una serie de instrucciones simples que, ejecutadas en el orden correcto, permiten realizar una tarea determinada.

```
*** Ejecución Iniciada. ***
Ingrese su Nombre:
> Pablo
Hola Pablo
*** Ejecución Finalizada. ***
```

Variables

Para que un programa pueda procesar información, primero es necesario **almacenar los datos con los que va a trabajar**. De esta manera, el programa puede acceder a esos valores cuando los necesite para realizar cálculos, comparaciones u otras operaciones.

Para cumplir esta función, los programas utilizan **variables**, que permiten guardar y manipular información durante la ejecución del algoritmo.

Una **variable** es un espacio en memoria que se utiliza para **almacenar información durante la ejecución de un programa**. Este espacio permite guardar datos que pueden ser utilizados o modificados a lo largo del algoritmo.

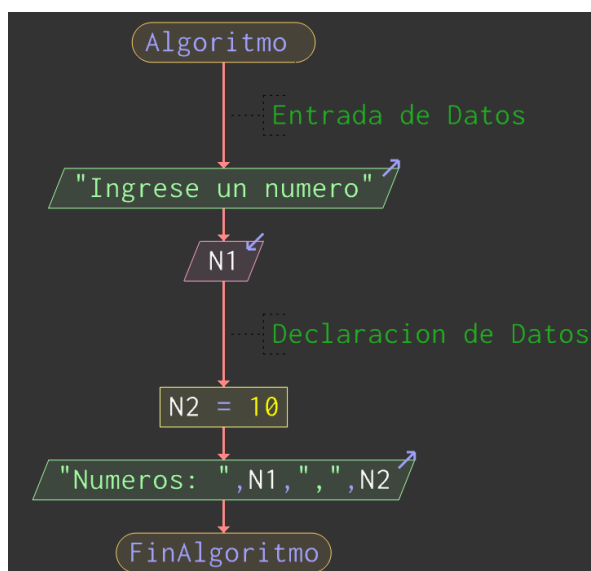
Cada variable posee un **nombre**, que permite identificarla dentro del programa, y un **valor**, que corresponde al dato que se encuentra almacenado en ella en un momento determinado.

Las variables presentan algunas características importantes:

- **Identificación:** cada variable tiene un nombre que permite referenciarla dentro del programa.
- **Almacenamiento de datos:** permiten guardar información que será utilizada durante la ejecución del algoritmo.
- **Modificación:** el valor almacenado puede cambiar a medida que el programa se ejecuta.

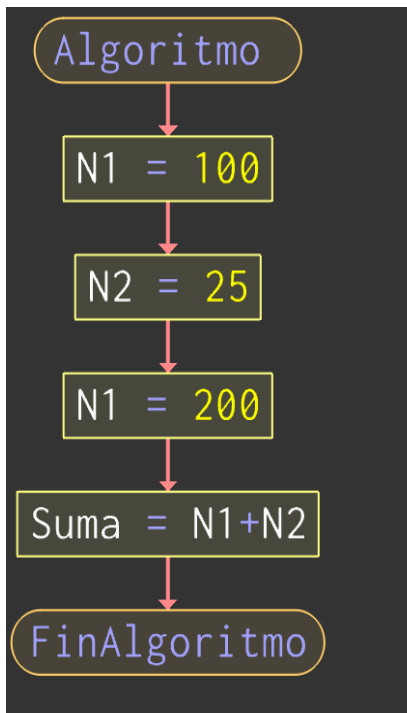
El valor asignado a una variable puede ser **un dato concreto**, como un número o un texto, o bien **el resultado de un proceso o cálculo realizado por el programa**.

En un diagrama de flujo, las variables pueden obtener su valor de distintas maneras. Una variable puede **recibir un valor ingresado por el usuario mediante el teclado**, a través de una instrucción de entrada de datos. También puede **obtener su valor como resultado de un proceso**, es decir, a partir de una asignación directa, una operación o un cálculo realizado dentro del algoritmo. De esta manera, las variables pueden almacenar tanto datos proporcionados por el usuario como resultados generados durante la ejecución del programa.



```
*** Ejecución Iniciada. ***
Ingrese un numero
> 25
Numeros: 25,10
*** Ejecución Finalizada. ***
```

Ejemplo: Un programa que declare dos variables y las sume



- **Declaración de la variable N1:**

Se crea la variable **N1** y se le asigna el valor 100, reservando un espacio en memoria para almacenar ese dato.

- **Declaración de la variable N2:**

Se crea la variable **N2** y se le asigna el valor 25, quedando este valor almacenado en memoria.

- **Modificación del valor de una variable:**

Se asigna un nuevo valor a la variable n1, pasando de 100 a 200. Esto muestra que el valor de una variable puede cambiar durante la ejecución del programa.

- **Declaración de una variable a partir de una operación:**

Se crea la variable suma, cuyo valor se obtiene al realizar la operación $N1 + N2$. **El resultado de esta operación se almacena en la variable suma.**

```
*** Ejecución Iniciada. ***
Numero 1: 100
Numero 2: 25
Suma: 225
*** Ejecución Finalizada. ***
```

Como puede observarse, el valor final de la variable **Suma** se obtiene al sumar el valor de **N2** con el **último valor asignado a N1**. Dado que **N1** pasó a valer **200** y **N2** vale **25**, el resultado almacenado en **Suma** es **225**.

Tipos de Datos

Al momento de diseñar un algoritmo, es frecuente que sea necesario trabajar con una gran cantidad de datos. Sin embargo, los datos que maneja un algoritmo no siempre poseen la misma naturaleza. En algunos casos se trabaja con **valores numéricos**, en otros casos con **cadenas de texto**, y en otros con **valores lógicos**.

Estas diferentes formas en que puede presentarse la información reciben el nombre de **tipos de datos**. Identificar el tipo de dato adecuado resulta importante para poder representar correctamente la información y operar con ella dentro de un algoritmo.

Algunos de los tipos de datos más utilizados a la hora de diseñar un algoritmo o programa son los siguientes:

- **Enteros:** Representan números **sin parte decimal**.
Por ejemplo:
 - 25
 - 0
 - -3

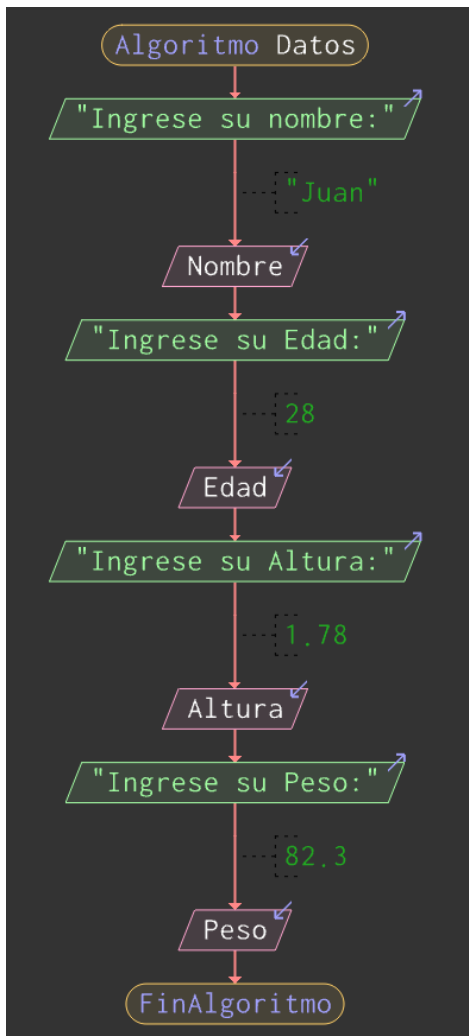
- **Reales:** Representan números que **poseen parte decimal**.
Por ejemplo:
 - 999.99
 - 0.032
 - -273.2

- **Cadenas de Texto:** Se utilizan para representar **texto**, es decir, letras, palabras o frases completas.
Por ejemplo:
 - A
 - Numero
 - Buenos Días

- **Lógicos o booleanos:** Representan valores de **verdad lógica**. Solo pueden tomar dos valores posibles:
 - Verdadero
 - Falso

Ejemplo: Un programa nos pide nuestros datos para un chequeo medico

La estructura seria la siguiente:



- **Ingreso del nombre**

El algoritmo solicita al usuario que ingrese su **nombre**. Este dato corresponde a una **cadena de caracteres**, ya que está compuesto por letras que forman una palabra o texto.

- **Ingreso de la edad**

Luego se solicita la **edad** del usuario. En este caso se trata de un **número entero**, ya que la edad generalmente se expresa mediante números sin parte decimal.

- **Ingreso de la altura**

A continuación se pide la **altura** de la persona. Este dato se representa mediante un **número real** ya que puede incluir valores con decimales, por ejemplo 1.78 metros.

- **Ingreso del peso**

Finalmente se solicita el **peso** de la persona. Al igual que la altura, este valor también **puede incluir decimales**, por lo que se representa mediante un **número real**.

```
*** Ejecución Iniciada. ***
Nombre: Juan
Edad: 28
Altura: 1.78
Peso: 82.3
*** Ejecución Finalizada. ***
```

Operadores aritméticos

Una vez comprendidos los conceptos de **variables** y **tipos de datos**, es posible comenzar a **procesar la información dentro de un algoritmo**. En muchos casos, la resolución de un problema requiere realizar **operaciones matemáticas** sobre los datos disponibles, como sumar valores, calcular diferencias, obtener productos o realizar divisiones.

Para llevar a cabo estas operaciones se utilizan los **operadores aritméticos**, que permiten establecer relaciones matemáticas entre valores numéricos y generar nuevos resultados durante la etapa de procesamiento del algoritmo.

Los principales operadores aritméticos son los siguientes:

- **Suma (+)**

El operador de suma permite **combinar dos valores numéricos para obtener su resultado total**.

Ejemplo:

$$5 + 3 = 8$$

- **Resta (-)**

El operador de resta permite **calcular la diferencia entre dos valores numéricos**, restando un valor del otro.

Ejemplo:

$$10 - 4 = 6$$

- **Multiplicación (*)**

El operador de multiplicación permite **obtener el producto entre dos valores numéricos**

Ejemplo:

$$6 * 3 = 18$$

- **División (/)**

El operador de división permite **dividir un valor numérico por otro**, obteniendo como resultado un cociente. Dependiendo del tipo de datos utilizado, el resultado puede ser un número entero o un número con parte decimal.

Ejemplo:

$$12 / 3 = 4$$

$$14 / 4 = 3,5 \text{ si el tipo de dato es Real.}$$

- **Módulo o resto (%)**

El operador módulo permite **obtener el resto de una división entera**.

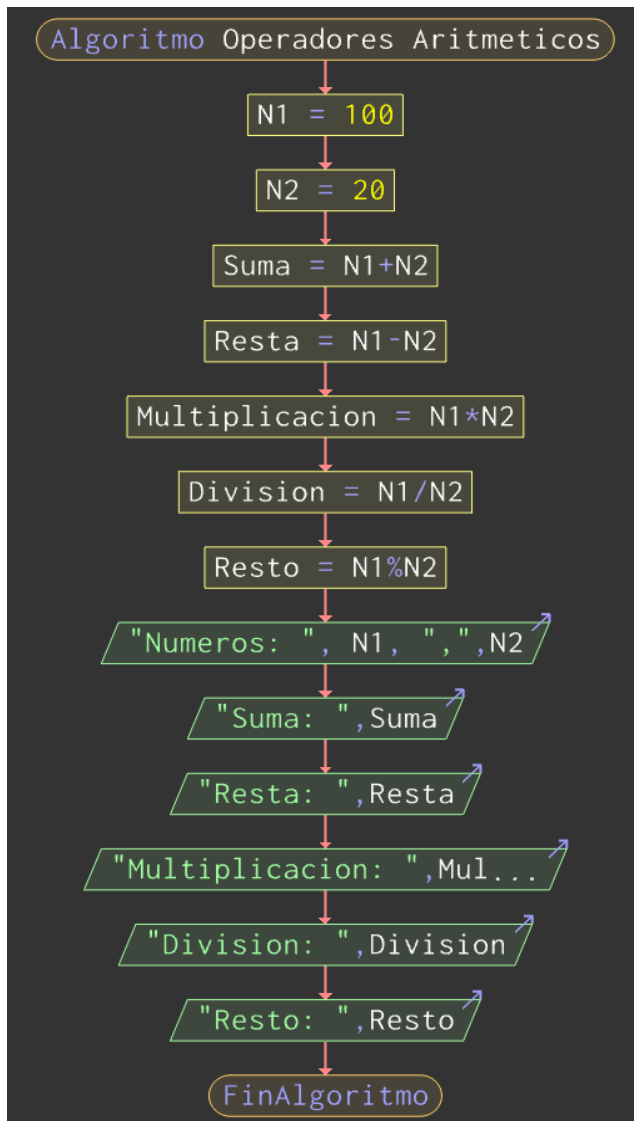
Este operador resulta especialmente útil en ciertos tipos de problemas algorítmicos, como la determinación de números pares o impares, ciclos periódicos o distribución de elementos en grupos.

Ejemplo:

$$10 \% 3 = 1$$

(ya que 3 entra tres veces en 10 y queda un resto de 1)

Ejemplo: Un programa que nos muestre el resultado de hacer una operación entre dos números con cada uno de los operadores aritméticos.



- **Declaración de la variable N1**
Se declara la variable **N1** y se le asigna el valor **100**.

- **Declaración de la variable N2**
Se declara la variable **N2** y se le asigna el valor **20**.

- **Cálculo de la suma**
Se declara la variable **suma**, cuyo valor corresponde al resultado de **sumar el valor de N2 al valor de N1**

- **Cálculo de la resta**
Se declara la variable **resta**, cuyo valor corresponde al resultado de **restar el valor de N2 al valor de N1**.

- **Cálculo de la multiplicación**
Se declara la variable **multiplicación**, cuyo valor corresponde al resultado de **multiplicar N1 por N2**.

- **Cálculo de la división**
Se declara la variable **división**, cuyo valor corresponde al resultado de **dividir N1 entre N2**.

```
*** Ejecución Iniciada. ***
Numeros: 100,20
Suma: 120
Resta: 80
Multiplicacion: 2000
Division: 5
Resto: 0
*** Ejecución Finalizada. ***
```

- **Cálculo del resto**
Se declara la variable **resto**, cuyo valor corresponde al **resto de la división entre N1 y N2**, utilizando el operador módulo.

- **Visualización de los resultados**
Finalmente, el algoritmo **muestra por pantalla el valor almacenado en cada una de las variables**, permitiendo observar el resultado de todas las operaciones realizadas